NightShift: Churn-Tolerant Batch Inference of Large Language Models on Pools of Charging Smartphones

Aidin Bazarchi

[a]*Independent Researcher,*

Abstract

Many smartphones spend the night plugged in, on Wi-Fi and idle. We study whether such a pool can run inference for a large language model (LLM) too big for any single phone, and what it would take. A 70-billion-parameter model at 4 bits needs about 40 GB, so one copy of the model is a pipeline of 16 phones, each of which leaves without warning when its owner unplugs it. We first derive what such a pool can and cannot do: it is a throughput machine for batch work with seconds per output token, not a conversational service. We then build a churn-aware simulator whose phone, network and user parameters are explicit, replaceable assumptions, and an analytical model of the data traffic that churn causes, which predicts the simulator's traffic within 12% across ten scheduling policies and pool sizes from 500 to 4,000 phones. Both show that once phones come and go, the dominant cost of churn is moving model shards to the phones that replace leavers, not recomputing lost work: a fair cache-aware baseline downloads 363 GB per million tokens, about 4.0 GB per serving phone per day. NightShift gives every phone a permanent home block, lets a vacant stage wait briefly for a phone that already holds it, and consolidates broken pipelines. With 1,000 phones it downloads 6–11× less data than each of six baselines, with throughput equal

to Petals-style replay within noise and 5% below the best baseline; at 4,000 phones it is ahead on both. Dedicated warm standbys, the usual remedy for failures, cut throughput by 39%. We discuss privacy, integrity, battery wear, data plans and incentives, report [Study A: measured per-layer times, batching cost, hop latency and session statistics from N Android phones], and release the simulator, the model and a benchmark kit.

Keywords: large language models, distributed inference, volunteer computing, smartphones, pipeline parallelism, churn, edge computing

---

1. Introduction

Volunteer computing has a long record. BOINC and SETI@home turned idle desktops into a scientific computer (Anderson, 2004; Anderson and Fedak, 2006), Folding@home did the same for protein dynamics (Shirts and Pande, 2000), and Google's federated learning system trains models on phones only while they are "idle, charging and on an unmetered network" (Bonawitz et al., 2019). Large language models now consume a large and growing share of data-centre compute (Samsi et al., 2023; Luccioni et al., 2024), which invites a question: could the idle phones of the world run them?

The question has an easy wrong answer in each direction. "Yes, enough phones beat any data centre" ignores that generation is sequential and that a large model does not fit in a phone. "No, phones are too small" ignores that the same was said of volunteer desktops, and that systems such as Petals (Borzunov et al., 2023a,b) already serve 100B-class models from pipelines of volunteer GPUs over the internet. The interesting question is quantitative: *for which workloads, at what throughput and at what cost to the*

volunteers, and which design choices matter?

Three properties make a pool of phones different from the volunteer GPUs that Petals targets and from the preemptible cloud instances that SpotServe (Miao et al., 2024) and Bamboo (Thorpe et al., 2023) target. A phone holds a small slice of a large model, so even one model copy spans many devices. Phones leave without warning, at a rate that follows their owners' daily lives. And every byte moved to a phone is paid for by its owner, in data, time and battery, so data movement is not a free internal cost of the scheduler but the main thing a volunteer gives up. The last property, we find, decides which designs are viable.

This paper makes four contributions.

1. **A first-principles analysis** (Section 2) of what a phone pool can and cannot do. It suits throughput-oriented batch jobs, not interactive chat, and it reads far more weight memory per token than a GPU, so its case rests on marginal cost, not efficiency.
2. **A churn-aware simulator** (Section 7) driven by a diurnal model of when phones are charging and on Wi-Fi. Every phone, network and user parameter is explicit and can be replaced by a measurement; a benchmark kit produces those measurements on Android phones (Section 9).
3. **An analytical model of shard traffic** (Section 6) that explains where the data a pool downloads comes from. It treats each scheduling decision as a draw from the idle pool and predicts the simulator's downloads within 12% for ten policies at 1,000 phones and for four of them at pool sizes from 500 to 4,000. It shows why matching cached phones, however

carefully, cannot remove most of the traffic when phones keep whatever block they served last, and why a fixed assignment can.

4. **NightShift** (Section 5), a placement and recovery design built on that observation: a permanent home block per phone, a short wait for a phone that holds the vacant block, and consolidation of broken pipelines. It cuts data downloaded 6–11× against six baselines at 1,000 phones, including a fair cache-aware one, and 5× against cache-aware even when phones cache the whole model. Throughput matches replay within noise and is 5% below the best baseline at this size; NightShift overtakes that baseline as the pool grows (from about 2,000 phones) and trails at 500 phones. An ablation shows that neither the wait nor consolidation helps alone, and that dedicated standbys make things worse.

We also examine the questions that decide whether such a system should be deployed at all: the privacy of prompts passing through strangers' phones, the integrity of their outputs, battery wear and heat, data plans, and incentives (Section 10 and Section 11). We are explicit about what is measured and what is assumed. The design results come from simulation with stated assumptions (Table 3); Section 9 reports the measurements that calibrate them and quantifies how far the simulator's timing model is from a real pipeline of phones. The simulator, the analytical model, all results and the benchmark kit are released (see Data availability).

2. The physics of a phone pool

This section derives, from a few numbers, the operating regime of a pool of phones. Table 3 lists the values; Section 9 replaces them with measurements.

Memory. A 70B model quantised to about 4 bits per weight (Frantar et al., 2023; Lin et al., 2024) occupies roughly 40 GB; we model 80 layers of 0.5 GB, the shape of Llama-class models (Touvron et al., 2023; Grattafiori et al., 2024). An Android app running in the background can rarely hold more than a fraction of a phone's RAM; we assume 45%, so an 8 GB phone offers 3.6 GB. With the key–value (KV) cache for a batch of 32 sequences of 768 tokens, a five-layer block needs 3.0 GB, so a model copy is a pipeline of $B = 16$ phones, and phones with 6 GB of RAM cannot take part at all (90% of our pool is eligible). Smaller blocks would admit more phones but lengthen the pipeline, which raises both the time per token and the chance that some member leaves.

Time per output token. Generating one token visits all B stages in order. Let τ_i be the compute time of stage i for one micro-batch and h the one-way latency of a hop between phones. A token's *trip* through the pipeline takes $\sum_i \tau_i + Bh$. With B micro-batches in flight, a pipeline also cannot finish a round of tokens faster than B times its slowest stage. The time per output token is therefore

$$T_{\text{tok}} = \max\Big(\sum_{i=1}^{B} \tau_i + Bh,\ B \max_i \tau_i\Big). \tag{1}$$

At roughly 15 ms per layer on a mid-range phone (plus 10% for micro-batches of two) and 60 ms per wide-area hop, the trip takes about 1.3 s of compute plus 1.0 s of network, and the slowest stage bound is typically about 2 s. Equation (1) gives 2.3 s at 60 ms hops, where the trip dominates, and still 2.1 s on a household network (5 ms hops), where the slowest phone sets the pace and the trip alone would take only 1.4 s. Speculative decoding (Leviathan et al., 2023) or faster phones would shorten this, but not to the sub-second

times a conversational assistant needs.

Throughput. Latency need not limit throughput. With many sequences in flight, a pipeline keeps every stage busy, as in the micro-batching of GPipe and PipeDream (Huang et al., 2019; Narayanan et al., 2019) and in continuous batching servers (Yu et al., 2022; Kwon et al., 2023; Agrawal et al., 2024). A pipeline of B stages that carries S sequences produces S/T_{tok} tokens per second, about 13 tokens per second for $S = 32$ at our parameters, before losses to churn. Batch workloads such as offline evaluation, synthetic data generation, document summarisation or embedding a corpus tolerate hours of latency, the same regime FlexGen targets on a single GPU (Sheng et al., 2023), and providers already sell such work at a discount through batch interfaces.

Memory traffic and energy. The price is efficiency. A data-centre GPU serving a batch of 128 reads each weight once per 128 tokens, about 0.3 GB of weight traffic per token for a 70B model. A phone pipeline with 32 sequences over 16 stages processes micro-batches of two, so it reads about 20 GB per token, roughly 60× more. At an assumed 4 W per active phone and 1 W while waiting, the simulated pool spends about 6 J per token, which is well above published per-token energy figures for GPU servers (Samsi et al., 2023), although those come from different models and workloads and are not a like-for-like comparison. The phone pool's argument is therefore not efficiency but *marginal cost*: the hardware is already bought, already powered at night and otherwise idle. Whether that argument survives battery wear and the owner's electricity bill is examined in Section 11.

Data movement. The last number is the one this paper is about. A block is 2.5 GB. At a median residential downlink of 50 Mbit/s, moving one block to a phone takes about 7 minutes; reading it from the phone's own flash takes 2.5 s. A phone that has to download a block whenever it is recruited for a different one pays minutes of idle time and gigabytes of its owner's data each time. How often that happens is a property of the scheduler, and it is what Section 6 models.

3. Related work

Volunteer, desktop-grid and federated computing. BOINC (Anderson, 2004) established redundant execution and work units for untrusted volunteers, and its projects measured the computational and storage potential of volunteer hosts (Anderson and Fedak, 2006); surveys summarise the requirements such systems share (Durrani and Shamsi, 2014). Desktop-grid studies characterised host availability and its effect on scheduling (Kondo et al., 2004), fitted statistical models of availability to SETI@home hosts (Javadi et al., 2011), and collected failure traces from many distributed systems (Kondo et al., 2010). These systems ran independent jobs: a host that left lost only its own work unit. Federated learning (McMahan et al., 2017; Bonawitz et al., 2019; Kairouz et al., 2021) runs on phones at production scale and established the eligibility rule we adopt (charging, on Wi-Fi, idle). Participant selection (Lai et al., 2021) and asynchronous aggregation (Huba et al., 2022) deal with heterogeneous, intermittently available clients; device availability has been measured at large scale (Yang et al., 2021; Lai et al., 2022), and those traces are natural inputs to our simulator. In federated learning each phone trains

a full small model; nothing is split across devices, so a departure costs one update, not a pipeline.

LLMs over unreliable or heterogeneous hardware. Petals (Borzunov et al., 2023a,b) serves large models from pipelines of volunteer GPUs and recovers from departures by routing around them and replaying the lost stage's cache; it is the baseline we call *replay*. SWARM (Ryabinin et al., 2023), Learning@home (Ryabinin and Gusev, 2020) and DiLoCo (Douillard et al., 2023) train over unreliable or poorly connected nodes. Deserve (Wu et al., 2025) studies decentralised serving on volunteer GPUs; HexGen (Jiang et al., 2023) and Helix (Mei et al., 2025) place LLM inference over heterogeneous GPUs. SpotServe (Miao et al., 2024) serves LLMs on preemptible cloud instances, migrating context when an instance is reclaimed, and SkyServe (Mao et al., 2025) spreads replicas across regions and clouds to survive spot preemptions. For training, Bamboo (Thorpe et al., 2023) adds redundant computation and Oobleck (Jang et al., 2023) keeps redundant pipeline templates to survive failures, and Parcae (Duan et al., 2024) predicts spot-instance availability to adapt parallelism proactively, the same idea as our lifetime grouping. Those nodes are GPUs with tens of gigabytes and, often, seconds to minutes of warning; ours are phones with a few gigabytes and no warning, and moving a shard to a replacement costs the volunteer data.

Parallelism for LLM serving. Model and pipeline parallelism (Shoeybi et al., 2019; Huang et al., 2019; Narayanan et al., 2019) and their automatic combination (Zheng et al., 2022) are the basis of every multi-device LLM system. Serving systems batch continuously (Yu et al., 2022), page the KV

cache (Kwon et al., 2023), split prefill from decode across machines (Patel et al., 2024; Zhong et al., 2024), chunk prefills to balance latency and throughput (Agrawal et al., 2024), and partition large models carefully across accelerators (Pope et al., 2023; Aminabadi et al., 2022). They assume a stable set of well-connected accelerators. We borrow their pipeline arithmetic and keep their batching, and change the problem: membership changes all the time.

Splitting models across nearby devices. LinguaLinked (Zhao et al., 2023) runs one LLM across several Android phones, Galaxy (Ye et al., 2024) and EdgeShard (Zhang et al., 2024) split transformers across edge devices, exo (exo labs, 2024) does so for a household's laptops and phones, and prima.cpp (Li et al., 2026) runs 70B-class models on a home cluster of everyday devices; a recent survey covers this space (Qu et al., 2025). Earlier, MoDNN (Mao et al., 2017) and DeepThings (Zhao et al., 2018) split CNNs, MAUI (Cuervo et al., 2010) and CloneCloud (Chun et al., 2011) offloaded phone code to the cloud, and CWC (Arslan et al., 2012) scheduled jobs on the idle, charging phones of an organisation. Table 1 places NightShift among the closest systems. They assume a small, stable set of devices, typically in one home or one local network. We study the opposite regime: a large pool in which membership changes all the time.

Single-phone inference. LLM in a flash (Alizadeh et al., 2024), PowerInfer-2 (Xue et al., 2024) and runtimes such as llama.cpp (Gerganov and contributors, 2023) and MLC LLM (MLC team, 2023) make single phones faster; measurement studies report their throughput, energy and thermal behaviour (Laskaridis et al., 2024; Li et al., 2024), and MobileLLM (Liu et al.,

Table 1: Closest systems. "Churn": devices leave during inference (for SpotServe, after a short preemption notice). "Shard traffic": the cost of moving weights to replacements is modelled.

System	Devices	Churn	Shard traffic
Petals (Borzunov et al., 2023b)	volunteer GPUs	yes	no
SpotServe (Miao et al., 2024)	cloud spot GPUs	with warning	no
LinguaLinked (Zhao et al., 2023)	phones, one LAN	no	no
EdgeShard (Zhang et al., 2024)	edge devices	no	no
prima.cpp (Li et al., 2026)	home devices	no	no
CWC (Arslan et al., 2012)	charging phones	yes	n/a (small jobs)
NightShift (this work)	charging phones	yes	yes

2024) designs models that fit one phone. These are complementary: a faster phone shortens every stage, and a model that fits one phone needs no pipeline at all, which is the right choice whenever such a model is good enough.

Simulation of distributed systems. Simulators such as SimGrid (Casanova et al., 2014), CloudSim (Calheiros et al., 2011) and iFogSim (Gupta et al., 2017) model resource management in grids, clouds and fog deployments. Our simulator is deliberately narrower: it models one workload (a pipelined LLM batch), one resource class (phones that come and go) and the two costs that matter for it (lost work and shard movement), which keeps every assumption visible and lets a reader replace each with a measurement.

4. System model and problem

Phones and sessions. A pool has N phones. Phone j has usable memory m_j, a per-layer decode time ℓ_j and a downlink rate d_j. It is *available* during charging sessions $[s, e)$ in which it is plugged in, on Wi-Fi and idle; it can join the pool at s and leaves at e without warning. A phone is *eligible* if m_j can hold one block and its KV cache. Each phone keeps c blocks in flash between sessions (we assume $c = 1$, 2.5 GB).

Model and work. The model is cut into B blocks of consecutive layers, the *shard catalogue*. A *pipeline* assigns one eligible phone to each block and advances a batch of S sequences in lock step, round by round, following Equation (1). Work arrives as a queue of batch jobs that is never empty (we study capacity, not queueing).

Events. A coordinator acts in discrete ticks. When a member of a pipeline leaves, its stage is vacant and its sequences pause. The coordinator may (i) form new pipelines from idle phones, (ii) fill a vacant stage with an idle phone, (iii) dissolve a pipeline, returning its phones to the idle pool, and (iv) keep idle phones as standbys of a stage. A phone recruited for block b loads it from flash if it holds b and otherwise downloads it. A replacement must also rebuild the KV cache of its stage, by replaying the tokens so far (we transfer the stage's input activations to it) or by restarting the batch.

Objective. We want high throughput, output tokens per second per 1,000 phones, and low traffic, gigabytes downloaded per million output tokens, counting both shards and repair activations. Traffic is the cost the volunteers bear; throughput is what the pool is for. The two conflict: waiting for a

phone that holds the right block saves a download and costs time. Because the coordinator does not know when phones will leave, it can only predict; a policy is a rule for (i)–(iv) given the idle pool, the vacant stages and those predictions.

5. NightShift

NightShift has three parts and a recovery rule, summarised in Figure 1.

Sticky shards. Every phone is assigned a *home block* once and keeps it in flash storage indefinitely. New pipelines are formed only from complete sets of idle phones, one per home block, so forming a pipeline never downloads. A vacant stage is filled by an idle phone whose home block matches when there is one; loading from flash takes seconds, whereas downloading 2.5 GB over a residential link takes minutes and costs the owner data. If no matching phone is idle, the vacant stage waits up to $W = 300\,\mathrm{s}$ before falling back to any idle phone, which must download the block; a pipeline whose stage has been vacant for ten minutes is dissolved and its phones return to the pool (this rule applies to every policy). Home blocks are assigned round-robin when a phone joins, so each block has about the same number of owners.

Lifetime-grouped pipelines. A pipeline lives only as long as its shortest-lived member. The coordinator predicts when each phone's charging session ends, from that user's history of sessions starting at a similar time of day, sorts each block's idle phones by predicted end, and builds the i-th pipeline from the i-th phone of every block. Phones expected to stay until morning end up together, and short daytime sessions are grouped with each other instead of cutting long-lived pipelines short.

Consolidation. In the morning, phones leave faster than idle phones arrive, leaving many pipelines with one or two vacant stages that nobody can fill. Waiting wastes every phone in those pipelines. The rule runs at every tick, after repairs: when at least two pipelines still have a vacant stage, NightShift dissolves the most damaged one (most vacancies, then least progress) and lends its phones, which already hold blocks of the same catalogue, to the others. A pool that shrinks keeps as many *complete* pipelines as it can afford. Because the donor's phones hold the blocks the survivors are missing, consolidation is also what keeps the wait short: without it, a vacant stage often waits for a home-block phone that is not coming. Section 6 makes this precise.

Recovery. A replacement phone recomputes the KV cache of its block for the tokens processed so far, as in Petals. To do so it needs the input activations of its block for every token so far (prompt and generated). We assume the previous stage keeps them and sends them over the replacement's downlink, 16 KiB per token per sequence (8,192 hidden dimensions in fp16); this transfer takes time and is counted as traffic for every policy that replays. We compare replay with restarting the batch, which sends no activations but loses the batch's progress.

What we chose not to do. Keeping k warm standbys per stage turns a failure into a fast swap but takes kB phones out of service per pipeline. We evaluate it as a baseline. We also do not let a phone cache several blocks by default: Section 8 shows that even caching the whole model does not let the baselines catch up, and 40 GB is a large request of a volunteer's storage.

Every tick t of the coordinator

1. **Departures.** For every pipeline member that is no longer available, mark its stage vacant.
2. **Repair.** For every vacant stage of block b: if an idle phone with home block b exists, recruit the one with the longest predicted stay (flash load); else, if the stage has waited less than W, wait; else recruit the idle phone with the longest predicted stay (download). Then transfer the stage's input activations and replay its KV cache.
3. **Consolidate.** While two or more pipelines have a vacant stage, dissolve the one with most vacancies (then least progress), return its phones to the idle pool and repeat step 2 for the others.
4. **Dissolve starved.** Dissolve any pipeline that has had a vacant stage for more than 600 s.
5. **Form.** While every block has an idle home-block phone, sort each block's idle phones by predicted stay and form a pipeline from the first phone of every block.
6. **Run.** Advance every complete pipeline by one tick of Equation (1).

Figure 1: The NightShift coordinator. Baselines differ in steps 2, 3 and 5 only (Section 7).

6. An analytical model of shard traffic

Why does a cache-aware scheduler still download so much, and why does a fixed assignment not? This section answers with a model that predicts, from a few counts, how many blocks each policy downloads. It makes one assumption:

(U) Each idle phone holds one block, uniform over the B blocks

and independent of the other idle phones.

For the random policies, (U) holds approximately because a phone keeps only the block it served last ($c = 1$) and that block was chosen by earlier, unrelated decisions. For NightShift it does not hold, and the model shows what replaces it. Let $q = 1 - 1/B$.

Repair from the whole pool. A repair for block b that may choose any of I idle phones, preferring one that holds b, downloads only if none does:

$$P_{\mathrm{dl}}^{\mathrm{repair}}(I) = q^I \approx e^{-I/B}. \tag{2}$$

With $B = 16$, an idle pool of 16 phones misses 35.6% of the time and one of 48 phones still misses 4.5% of the time. At night the idle pool is small, because most available phones are already serving, so repairs usually download. Ranking by predicted stay and looking only at the top quarter of the pool, as our lifetime baseline does, raises the exponent's denominator: $P = q^{\lfloor I/4 \rfloor}$.

Formation from a random group. The replay family forms a pipeline from a random group of $B(1 + k)$ idle phones and lets each stage, in turn, take a group member that holds its block or else the first remaining member. Even a perfect matching within the group would leave the expected number of blocks held by nobody, $Bq^{B(1+k)}$, to be downloaded: 5.7 of 16 for $k = 0$. The greedy order loses more, because a fallback may consume the only phone that holds a later block. We compute the expected downloads per formation with a mean-field recursion over the take order (the expected number of remaining phones holding each block, updated after each take): 7.6 of 16 blocks for $k = 0$.

Formation matched over the whole pool. The cache-aware baseline forms $n = \lfloor I/B \rfloor$ pipelines from an idle pool of I phones and gives each block to phones that hold it first. With X_b the number of idle phones holding block b, $X_b \sim \text{Binomial}(I, 1/B)$ under (U), and the expected downloads are

$$D^{\text{form}}(I) = \sum_{b=1}^{B} \mathbb{E}\big[\max(0,\, n - X_b)\big], \tag{3}$$

which is exact under (U). The fluctuation of X_b around its mean n is about $\sqrt{n}$, so a fraction of order $1/\sqrt{n}$ of the stages download even with perfect matching: about 12% at $n = 10$. Matching is limited not by the scheduler's cleverness but by the multinomial spread of what the idle phones happen to hold, which is why cache-aware helps less than expected (Section 8).

Sticky formation and repair. NightShift forms pipelines only from complete home-block sets, so formation does not download unless a phone has *drifted*: after serving as a fallback for another block, it holds that block instead of its home block, and its next use in its home role downloads again. A repair falls back when no home-block phone for b is idle when the stage becomes vacant and none joins the idle pool during the remaining wait W'. Two facts shape this probability. First, the idle pool under sticky formation is the *remainder* of a partition into complete sets: if every block had an idle owner, another pipeline would have formed. Conditioning the uniform pool on "some block is absent" gives

$$\begin{aligned} P_{\text{none}}(I_0) &= \frac{q^{I_0}}{P(\exists b : X_b = 0)}, \\ P(\exists b : X_b = 0) &= \sum_{j=1}^{B} (-1)^{j+1} \binom{B}{j} \Big(1 - \frac{j}{B}\Big)^{I_0}, \end{aligned} \tag{4}$$

which tends to $1/B$, not to 0, as the pool grows. Second, phones join the idle pool at a rate $a(h)$ that depends on the hour h: new sessions, and the phones released when a pipeline is dissolved. A fraction $1/B$ of them own b, so

$$P_{\text{fb}} = P_{\text{none}}(I_0)\, \exp\bigl(-a(h)\, W'/B\bigr). \tag{5}$$

Let $F = \sum P_{\text{fb}}$ be the expected number of fallbacks and H the number of uses of phones in their home role (formation stages plus home repairs). In steady state, fallbacks create drift at rate F and home uses of drifted phones remove it, so the drifted fraction is $\delta = F/(F + H)$, and the expected downloads are $F + H\delta$.

Equation (5) explains the ablation. Consolidation raises $a(h)$ exactly when it is needed: at dawn, new sessions are rare but dissolved pipelines release complete sets of home-block phones, so a vacant stage's wait is usually rewarded. Without consolidation the wait mostly expires; without the wait ($W' = 0$) every absent home block is a download. Only together is F small.

Check against the simulator. The model takes as input only what a scheduler observes: the idle-pool size at each decision (I, I_0), the number of formations and decisions, and the hourly join rate $a(h)$. It never sees which block any phone holds. We instrumented the simulator to log every assignment and compared predicted with simulated downloads. Table 2 gives the result. Across the ten policies at 1,000 phones and the four main policies at 500, 2,000 and 4,000 phones (5 seeds at 1,000, 3 elsewhere), the predicted traffic is within 12% of the simulated traffic, with a mean absolute error of 5.6%. The largest errors are for NightShift at 2,000 phones, where downloads are rare (about 99 per run), so a handful of events moves the relative error by

several points; among the busier policies the largest error is for Sticky, where the drift balance treats every fallback as equally likely to land on a phone that has not drifted yet.

What the model implies. Three consequences follow. First, with one cached block per phone, the replay family cannot get much below a few downloads per formation and $e^{-I/B}$ per repair, whatever the scheduler does, because the idle phones' blocks are spread over all B blocks. Second, a fixed home block turns the problem from "does some idle phone happen to hold b?" into "does an owner of b arrive within W?", which a larger pool, a longer wait or consolidation all make likely; this is why NightShift's traffic falls with pool size and the baselines' does not (Figure 5). Third, the model gives an operator a way to set W without simulation: choose the smallest W for which $\exp(-a(h)W/B)$ is small at the hours when repairs happen.

7. Simulation methodology

Simulator. The simulator advances in 10-second ticks. Each phone has a RAM class, a per-layer decode time and a downlink speed drawn from Table 3, and a sequence of charging sessions drawn from the diurnal model, or read from a real trace. A round of a pipeline takes T_{tok} from Equation (1) and produces one token for each of the 32 sequences. A micro-batch of b tokens costs $1 + 0.1(b - 1)$ times a single token, and prefill costs 1/8 of a decode token. A download delays the stage that waits for it by the block size over the phone's downlink; a repair also waits for the activation transfer and the replay.

Steady state. Seven days of synthetic history train the session-end predictor. The simulator then runs two days under the policy being evaluated and discards them, so that flash caches and pipelines reach that policy's own steady state, and finally measures 24 hours from noon. Every phone starts the warm-up holding one block (its home block), so a policy cannot look good merely by inheriting another's caches. Onboarding traffic, a phone's first download, is outside the measured window for every policy; a new volunteer downloads one block once.

Policies. We compare ten policies that differ only in placement and recovery. The baselines are *restart* and *replay* (random placement; restart a broken batch, or replay its cache as Petals does). Both already reuse caches where it is easy: a pipeline is formed from a random group of idle phones, each stage taking a phone in the group that holds its block if there is one, and a repair takes any idle phone that holds the block before falling back to a download. *Cache-aware* is replay with cache-aware formation: it matches each block to idle phones that hold it across the whole idle pool, and only then fills the remaining stages from the rest (it never waits); *standby* (replay plus one warm standby per stage); *lifetime* (replay with pipelines grouped by predicted session end); and *consolidate* (replay plus consolidation). The NightShift ablation adds one part at a time: *wait* (home blocks and the five-minute wait, random order), *sticky* (plus lifetime grouping), *NightShift without the wait* (sticky plus consolidation, falling back to a download at once) and *NightShift*. The *upper bound* places every available, eligible phone in a complete pipeline instantly and never loses work.

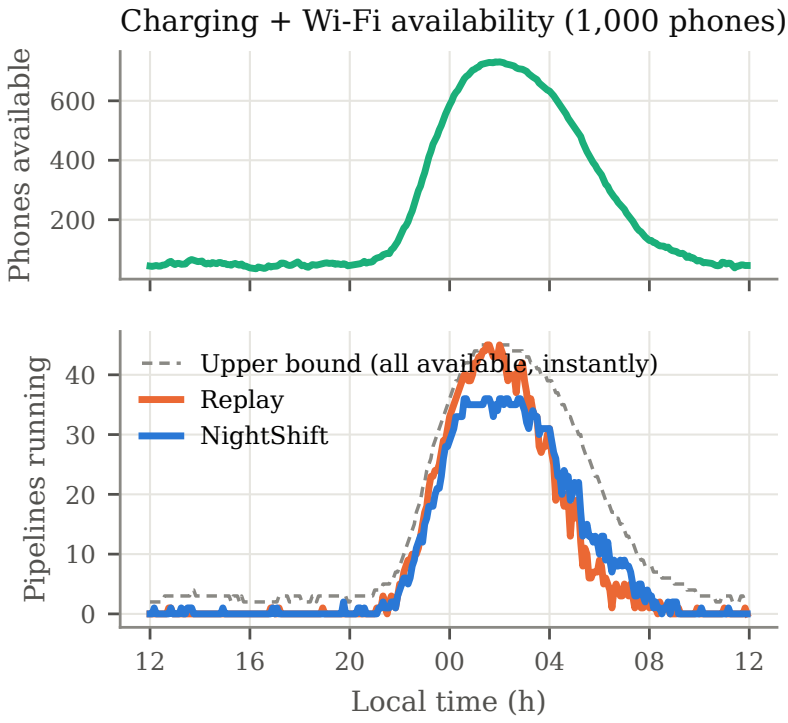


Figure 2: One simulated day. Top: phones available. Bottom: pipelines running and an upper bound that assumes instant, perfect placement.

Metrics and statistics. Throughput is output tokens per second per 1,000 phones in the pool. Traffic is gigabytes downloaded per million output tokens: shards plus repair activations. Results are means over 5 seeds with 1,000 phones, for the main comparison and for every sweep; differences between policies are paired by seed, so both see the same phones and sessions, and we report them with 95% t confidence intervals.

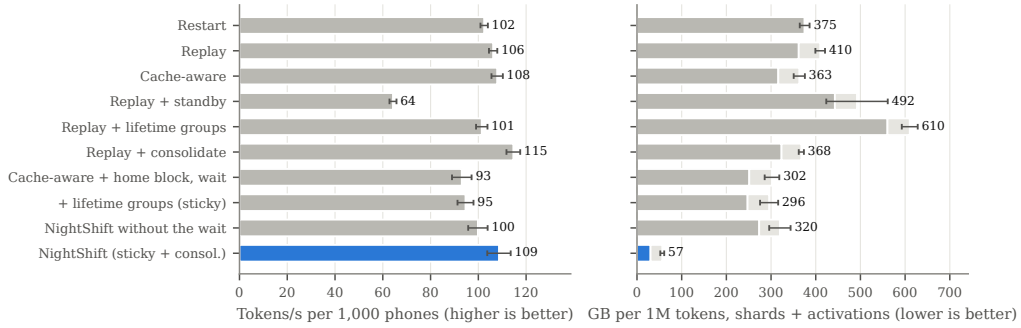


Figure 3: Throughput and data downloaded (shards, dark; repair activations, light) of the ten policies (1,000 phones, mean ± s.d. over 5 seeds). The last four rows add NightShift's parts one at a time.

8. Results

The pool is a night-time machine. Availability swings from 35 to 731 of 1,000 phones over the day (Figure 2), and 98% of tokens are produced between 22:00 and 08:00. Daytime sessions are short and scattered, so few complete 16-phone pipelines form. A phone pool is best treated as a batch system that drains a queue overnight.

Moving shards, not lost work, is the cost of churn. Figure 3 compares the policies. Restart differs from replay by only -4% (±0.4%, 95% CI), so little computation is lost to departures. What separates the policies is data movement. Replay already prefers phones that hold the needed block, yet the pool downloads 410 GB per million tokens, of which 362 GB are shards. Matching cached phones across the whole idle pool when forming pipelines (cache-aware) helps less than one would expect, to 363 GB. As Section 6 shows, a phone that served one block last time is equally likely to be needed for any of the other fifteen next time, and at a given moment the few idle phones

seldom hold the right one; a serving phone still downloads about 4.0 GB a day, enough to make a volunteer's data plan the binding constraint.

NightShift. NightShift downloads 57 GB per million tokens: 7× less than replay, 6× less than cache-aware and 7× less than consolidation; for shards alone the reduction against replay is 12×. A serving phone downloads about 0.7 GB a day instead of 4.4 GB. Its throughput is 109 tokens/s per 1,000 phones against 106 for replay, a paired difference of +2.4 ± 5.1, which is not significant. The best baseline, consolidation, is faster: 115 tokens/s, a paired difference of -6.0 ± 4.0 tokens/s, so NightShift is 5% behind it at this pool size. NightShift suffers 46% fewer stage failures than replay, and its stages spend 83% of their time computing rather than waiting, against 64%. The trade is therefore 5% of throughput against the best baseline at 1,000 phones for a large cut in the data volunteers must download.

The ablation: the wait and consolidation need each other. Giving each phone a home block and making vacant stages wait for it (*wait*) cuts traffic only to 302 GB and changes throughput by -13.1 ± 3.5 tokens/s against replay: in the morning, stages wait for home-block phones that have left for the day. Adding lifetime grouping changes little (95 tokens/s, 296 GB). Adding consolidation but no wait (*NightShift without the wait*) recovers some throughput but keeps traffic at 320 GB, because every vacancy still triggers a download. Only together does the design work: consolidation supplies home-block phones from dissolved pipelines, so the wait is usually short and usually rewarded (Equation 5). Lifetime grouping alone, without home blocks, is worse than random (101 tokens/s, 610 GB), because ranking phones by predicted stay overrides any chance of reusing a cached block.

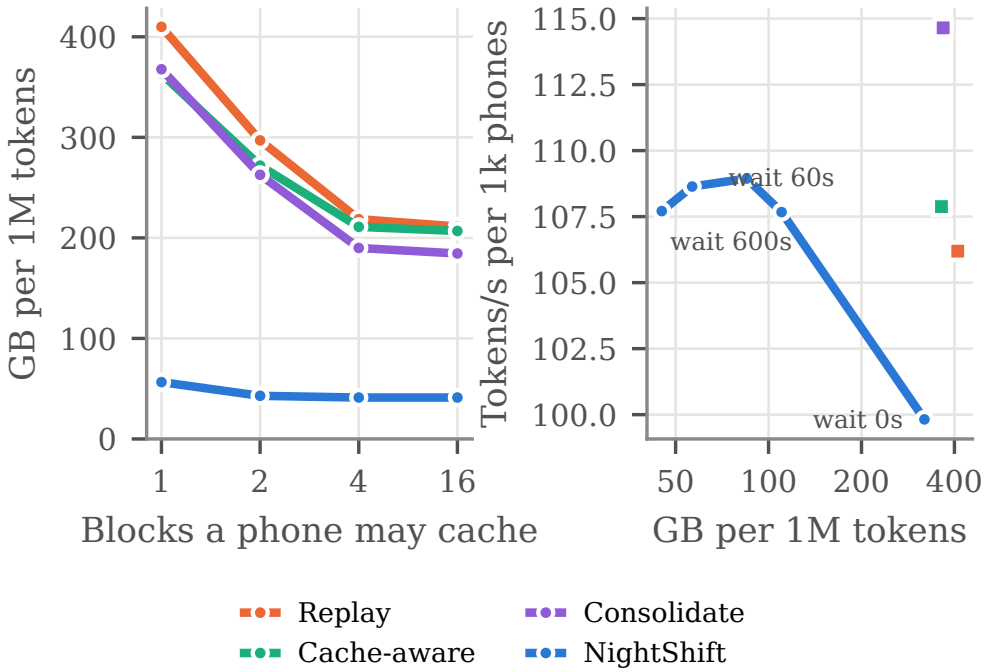


Figure 4: Left: traffic when each phone may cache more blocks. Right: throughput against traffic; NightShift's wait time (0–600 s) traces a trade-off, squares are baselines.

Standbys are a bad trade on phones. One standby per stage cuts throughput by 39%: it removes half of the pool from service to protect pipelines whose expected lifetime, at night, is hours. Redundancy pays off when failures are frequent relative to the cost of idling capacity, as for preemptible cloud GPUs (Thorpe et al., 2023; Jang et al., 2023), but not here.

Larger caches do not close the gap. One might expect the baselines to catch up if phones simply kept more blocks in flash. Figure 4 (left) varies the cache from one block to all 16 (the whole 40 GB model). Baseline traffic roughly halves (replay 211, cache-aware 207, consolidation 184 GB per million

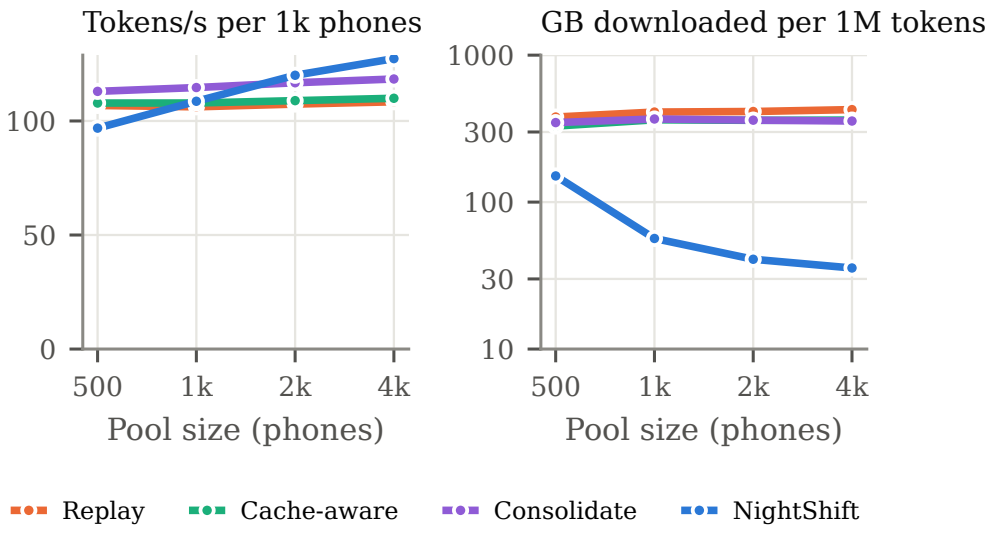


Figure 5: NightShift improves with pool size; the baselines do not. Traffic (shards plus activations) is on a logarithmic scale.

tokens with a full cache) while NightShift falls to 41 GB, still 5× less than cache-aware. With a full cache no shard is ever downloaded after onboarding, so the remaining traffic is repair activations, which NightShift's fewer failures also reduce.

The wait is a knob. Figure 4 (right) varies how long a vacant stage waits for a home-block phone. With no wait, NightShift is the no-wait ablation above (100 tokens/s, 320 GB). A 60-second wait already gives 108 tokens/s at 110 GB, and 600 s gives 108 tokens/s at 45 GB. Throughput is nearly flat beyond a minute while traffic keeps falling, so an operator can choose the point that suits the volunteers' data plans, using Equation (5) as a guide.

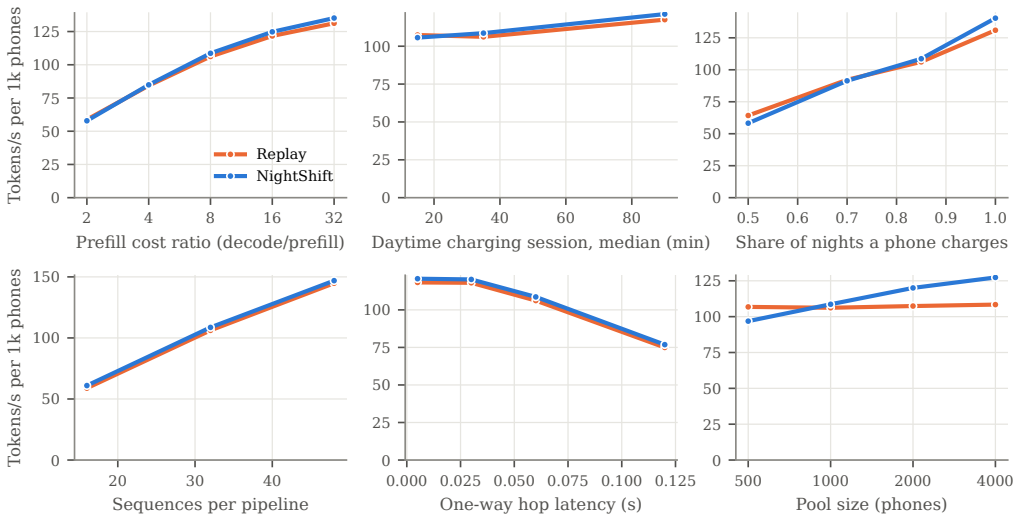


Figure 6: Sensitivity of throughput to the main assumptions, replay vs. NightShift (1,000 phones unless the panel varies pool size; mean of 5 seeds).

NightShift gets better with scale. With more phones, each block has more idle owners to choose from, so a vacant stage rarely waits long or falls back to a download (Figure 5). From 500 to 4,000 phones, NightShift's throughput rises from 97 to 127 tokens/s per 1,000 phones and its traffic falls from 151 to 36 GB per million tokens, while the baselines stay roughly flat (replay 107 to 108, consolidation 113 to 118). At 500 phones NightShift is worse: -9% against replay and -14% against consolidation, with only 3× less traffic. At 4,000 it is +17% against replay and +8% against consolidation, with 12× less traffic than replay and 10× less than cache-aware. A deployment worth building would have far more than 4,000 phones, but we have not simulated beyond that, and a large pool would likely be split by region (below), which brings each region back toward smaller pools.

Sensitivity. Figure 6 varies one assumption at a time. Three matter a lot. The share of nights a phone charges moves NightShift's throughput from 58 to 140 tokens/s per 1,000 phones between 50% and 100% of nights (at 50%, replay is slightly ahead). The batch size moves it from 61 (16 sequences) to 147 (48), because more sequences keep more stages busy; memory for the KV cache caps this. The cost ratio of decode to prefill moves it from 58 (ratio 2) to 135 (ratio 32): the prompt of every batch is processed once per stage, and each repair recomputes the cache of all tokens so far. Raising the median daytime session from 15 to 90 minutes changes throughput by +15% (106 to 121), less than the night share moves it, confirming that the night carries the system. Hop latency sets the time per output token once the trip is longer than 16 times the slowest stage (Figure 7: 2.1 s at 5 ms, where the slowest stage sets it, 2.3 s at 60 ms, 3.2 s at 120 ms), and with it throughput (121 to 77). Grouping pipelines by region, so that hops stay short, is therefore worth more than the scheduling differences we study. Across every setting we tried, NightShift downloads 3–12× less than replay, the low end at 500 phones. Its throughput differs from replay's by -9% to +17%: it is behind at 500 phones (-9%) and at 50% of nights (-9%), ahead at 2,000 and 4,000 phones (+17% at 4,000) and when every phone charges every night (+7%), and within about 4% elsewhere.

9. Measurements on real phones

The results above rest on assumed parameters. This section reports measurements that replace them, made with the released benchmark kit, and checks the simulator's timing model against a real pipeline of phones.

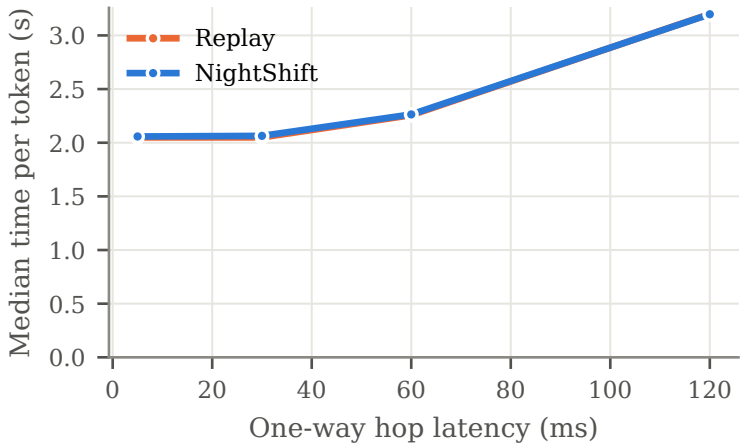


Figure 7: Median time per output token against one-way hop latency.

Setup. [Study A: list of phones (model, SoC, RAM, Android version), the 8B model and quantisation used, the network (same Wi-Fi / different homes), dates, and how many runs per measurement.] Compute is measured with llama.cpp (Gerganov and contributors, 2023) on Android (Termux): decode and prefill speed, the cost of batching 1–8 sequences, sustained speed over 30 minutes, and battery draw. Per-layer time is scaled to the 70B layer by bytes of weights read, since phone decode is memory-bandwidth bound. The pipeline test runs the 8B model split over 1–4 phones with llama.cpp's RPC backend.

Per-phone compute. [Table: measured ms per layer (70B-scaled) per phone vs. the assumed 10–30 ms; measured batching cost factor vs. the assumed $1 + 0.1(b - 1)$; measured prefill-to-decode cost ratio vs. the assumed 1/8.]

Sustained performance and heat. [Figure: tokens/s and battery temperature over a 30-minute run per phone; the ratio of sustained to initial speed;

measured power while computing and while idle-but-plugged vs. the assumed 4 W / 1 W.]

Network. [Measured one-way latency and throughput between two phones on the same Wi-Fi and in different homes, vs. the assumed 5 ms and 60 ms hops and 50 Mbit/s downlink.]

Timing model check. For a pipeline of p phones running the 8B model, Equation (1) predicts the time per token from the measured per-stage times and hop latency. [Table: predicted vs. measured time per token for $p = 1 \ldots 4$ and the relative error; target below 15%.]

Re-running the experiments. [Rerun every experiment with the measured parameters (PHONESIM_PARAMS) and report the headline numbers next to the simulated ones: throughput per 1,000 phones, GB per million tokens for NightShift and each baseline, and whether the ranking of policies changes.]

Availability. [Study B, if available: N volunteers' charging and Wi-Fi logs over 14 days, the fitted session distributions vs. the assumed ones, and NightShift vs. baselines on the real trace. Otherwise state that availability remains synthetic and cite large published traces (Yang et al., 2021) as the next input.]

10. Security, privacy and integrity

A pool of strangers' phones is an untrusted platform. We state a threat model, what NightShift does and does not address, and what deployment would need.

Threat model. The coordinator is trusted and run by the operator. Phones are run by volunteers, some of whom are curious (they inspect what passes through their stage), some faulty (they return wrong results because of bugs, overheating or corrupted weights), and a few malicious (they return wrong results on purpose, or join many times under different identities, a Sybil attack (Douceur, 2002)). Network links between phones may be observed. We do not consider attacks on the coordinator itself or denial of service.

Confidentiality of prompts. A stage sees the activations of the tokens that pass through it, and activations can be inverted to recover their input: model inversion attacks against split and collaborative inference (He et al., 2019; Gupta and Raskar, 2018) reconstruct inputs from intermediate representations, and the first stage, which sees embeddings, is the most exposed. Encrypting links protects against observers but not against the stage itself. Trusted execution environments can hide a computation from its host (Tramèr and Boneh, 2019), but phone TEEs are not available to ordinary apps for general computation at this scale. We therefore restrict the design to work whose inputs are not sensitive: public corpora, synthetic data generation, benchmark evaluation and similar batch jobs. This is a real limitation, and it also shapes the market: it excludes most enterprise data.

Integrity of results. A faulty or malicious stage can return wrong activations, and every later stage and every generated token depends on them. Three defences exist, at increasing cost. *Spot checks:* the coordinator recomputes a random sample of stage outputs on trusted hardware or on a second phone and compares; a phone caught once is excluded, so a cheater with probability p of being sampled per batch is caught within $1/p$ batches in expectation.

Lightweight fingerprints of the computation, such as TOPLOC (Ong et al., 2025), make the comparison cheap. *Redundant execution:* BOINC-style replication of each stage on two phones (Anderson, 2004) detects any single fault but halves throughput, which our standby result suggests is costly. *Proofs:* verifiable inference (Ghodsi et al., 2017) and zero-knowledge proofs for LLMs (Sun et al., 2024) give cryptographic guarantees but are orders of magnitude too slow for phones today. We expect spot checks to be the right first step, because the work is batch work whose results can be checked after the fact; Byzantine agreement (Lamport et al., 1982) is unnecessary when a trusted coordinator can re-run any stage.

Sybil and free-riding. If volunteers are rewarded, a participant may register many emulated phones. Hardware attestation of the device and its app, which both major mobile platforms offer, and rewards paid only for verified work raise the cost of such attacks. NightShift's home blocks help a little: a phone's identity is bound to a block it stores, so re-registering forces a new download that the operator can rate-limit.

Weights. Distributing a model to volunteers means that the weights leave the operator's control. This is acceptable for openly licensed models (Touvron et al., 2023; Grattafiori et al., 2024), which are the realistic first workload, and rules out proprietary ones unless each phone holds too small a slice to be useful, which a 2.5 GB block is not.

11. The owner's side: battery, heat, data and incentives

A volunteer system fails if volunteers leave. This section estimates what NightShift asks of an owner.

Battery wear and heat. Lithium-ion cells age faster at high temperature and at high states of charge (Vetter et al., 2005), and a phone that computes while charging overnight is both warm and nearly full. Users care about battery life and act on it (Ferreira et al., 2011; Banerjee et al., 2007), so a pool that shortens battery life will not keep its volunteers. Phones also throttle under sustained load (Bhat et al., 2019), which lowers a stage's speed over a night. Three rules limit the harm: compute only after the battery has reached a set level (charging then mostly tops up), pause when the battery temperature exceeds a threshold, and stop at a fixed share of the night. [Study A: measured battery temperature and power during a 30-minute run, and the fraction of the night a phone could compute under a 40 °C cap.] The simulator does not yet model these rules; they would lower throughput by the fraction of time phones pause.

Electricity. At the assumed 4 W while computing, a phone that serves for seven hours uses about 28 Wh a night, roughly 10 kWh a year, which at typical household tariffs costs a few euros a year. This is small for one owner and should be stated to volunteers plainly.

Data. This is where the design matters most. Under the cache-aware baseline a serving phone downloads about 4.0 GB a day, a heavy ask of any volunteer whose home connection has a data cap; under NightShift it downloads about 0.7 GB, plus one block (2.5 GB) when it joins. An operator should still cap each phone's monthly download and let volunteers set the cap; the wait time W (Figure 4) is the knob that keeps the pool within those caps.

Incentives. Why would anyone donate their phone's nights? Volunteer science has relied on altruism and recognition (Anderson and Fedak, 2006), which works for research but less well for commercial batch work. Incentive mechanisms designed for federated learning (Zhan et al., 2021) pay participants per contribution, and the same structure fits here: pay per verified stage-hour, with the payment funded by the discount at which batch inference is sold. Whether the marginal value of a phone's night exceeds its owner's costs (wear, data, electricity, attention) is an empirical question that a pilot with real volunteers would answer. We do not model incentives in the simulator; NightShift's contribution is to make the data cost small enough that the question can be asked.

12. Threats to validity

Construct validity. Our throughput metric counts output tokens and ignores the value of the jobs; our traffic metric counts bytes downloaded and treats all bytes as equal, while a volunteer may mind a download on mobile data far more than one on home Wi-Fi (we only use phones on Wi-Fi). Energy per token uses assumed power, not measured power.

Internal validity. The simulator ignores stragglers within a stage, thermal throttling over a night, Wi-Fi contention in one home, and iOS's stricter background limits. Downloads are counted and delay the stage that waits for them, but do not compete with other traffic on a link. Time advances in 10-second ticks, which coarsens short events. The analytical model of Section 6 was built from the simulator's rules, so its agreement with the simulator checks our understanding of the mechanism, not the simulator's

fidelity to real phones; that is the role of Section 9. We fixed the policies' timers (300 s wait, 600 s dissolution) before running the sweeps and report the wait sweep rather than a tuned value.

External validity. Each result comes from one synthetic 24-hour window per seed after warm-up, from a synthetic availability model; real traces may have more correlated departures (for example, everyone unplugging at 07:00 on a weekday) than ours, and different regions and seasons have different charging habits. We simulate up to 4,000 phones and one model size; other models change B and every number that depends on it. The phone population is a mix of RAM classes chosen to resemble mid-range Android devices, not a measured market share.

Conclusion validity. Policy differences are paired by seed and reported with 95% confidence intervals over 5 seeds; several throughput differences are within those intervals, and we say so where they are. We have not corrected for multiple comparisons across the ten policies.

13. Conclusion

A pool of charging phones can, in principle, run a model far larger than any phone, but only as an overnight batch system with seconds per output token and far more memory traffic per token than a GPU. Within that regime, the cost that matters most to the volunteers is data: replacing leavers moves model shards, and an analytical model shows why even a cache-aware design moves several gigabytes per phone per day when phones keep whatever block they served last. Giving each phone a permanent home block, waiting briefly

for a phone that holds it, and consolidating broken pipelines cut that traffic 6–11× against our baselines in simulation, at 5% of throughput against the best of them at 1,000 phones and a gain at 4,000; standbys hurt. Deployment would further need spot checks of results, restriction to non-sensitive inputs, limits on heat and data, and an incentive that covers the owner's costs. The next steps are measurements with real volunteers over weeks, larger simulated pools split by region, and a pilot with a small open model.

Data availability

The simulator, the analytical model, every result file, the figure scripts and the benchmark kit are available at `https://github.com/aidinb/nighshift-phone-llm` [and archived at Zenodo, DOI to be added at release].

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

CRediT authorship contribution statement

Aidin Bazarchi: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing – original draft, Writing – review & editing,

Visualization.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

[Author to confirm the wording.] During the preparation of this work the author used an AI assistant to help write and revise the text and the simulation code. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

References

Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B., Tumanov, A., Ramjee, R., 2024. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve, in: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), USENIX Association. pp. 117–134.

Alizadeh, K., Mirzadeh, I., Belenko, D., Khatamifard, K., Cho, M., Del Mundo, C.C., Rastegari, M., Farajtabar, M., 2024. LLM in a flash: Efficient large language model inference with limited memory, in: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), Association for Computational Linguistics.

Aminabadi, R.Y., Rajbhandari, S., Awan, A.A., Li, C., Li, D., Zheng, E., Ruwase, O., Smith, S., Zhang, M., Rasley, J., He, Y., 2022. DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale, in: SC22: International Conference for High Performance

684 Computing, Networking, Storage and Analysis, pp. 1–15. doi:10.1109/SC
685 41404.2022.00051.

686 Anderson, D.P., 2004. BOINC: A system for public-resource computing and
687 storage, in: Proceedings of the 5th IEEE/ACM International Workshop on
688 Grid Computing (GRID), IEEE. pp. 4–10.

689 Anderson, D.P., Fedak, G., 2006. The computational and storage potential of
690 volunteer computing, in: Sixth IEEE International Symposium on Cluster
691 Computing and the Grid (CCGRID'06), pp. 73–80. doi:10.1109/CCGRID
692 .2006.101.

693 Arslan, M.Y., Singh, I., Singh, S., Madhyastha, H.V., Sundaresan, K., Krish-
694 namurthy, S.V., 2012. Computing while charging: Building a distributed
695 computing infrastructure using smartphones, in: Proceedings of the 8th
696 International Conference on Emerging Networking Experiments and Tech-
697 nologies (CoNEXT), ACM.

698 Banerjee, N., Rahmati, A., Corner, M.D., Rollins, S., Zhong, L., 2007. Users
699 and batteries: Interactions and adaptive energy management in mobile
700 systems, in: UbiComp 2007: Ubiquitous Computing, Springer. pp. 217–234.
701 doi:10.1007/978-3-540-74853-3_13.

702 Bhat, G., Gumussoy, S., Ogras, U.Y., 2019. Power and thermal analysis
703 of commercial mobile platforms: Experiments and case studies, in: 2019
704 Design, Automation & Test in Europe Conference & Exhibition (DATE),
705 pp. 144–149. doi:10.23919/DATE.2019.8714831.

Bonawitz, K., Eichner, H., Grieskamp, W., Huba, D., Ingerman, A., Ivanov, V., Kiddon, C., Konečný, J., Mazzocchi, S., McMahan, H.B., Van Overveldt, T., Petrou, D., Ramage, D., Roselander, J., 2019. Towards federated learning at scale: System design, in: Proceedings of Machine Learning and Systems (MLSys), mlsys.org. ArXiv:1902.01046.

Borzunov, A., Baranchuk, D., Dettmers, T., Ryabinin, M., Belkada, Y., Chumachenko, A., Samygin, P., Raffel, C., 2023a. Petals: Collaborative inference and fine-tuning of large models, in: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (System Demonstrations), Association for Computational Linguistics.

Borzunov, A., Ryabinin, M., Chumachenko, A., Baranchuk, D., Dettmers, T., Belkada, Y., Samygin, P., Raffel, C., 2023b. Distributed inference and fine-tuning of large language models over the internet, in: Advances in Neural Information Processing Systems (NeurIPS), Curran Associates.

Calheiros, R.N., Ranjan, R., Beloglazov, A., De Rose, C.A.F., Buyya, R., 2011. CloudSim: A toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms. Software: Practice and Experience 41, 23–50. doi:10.1002/spe.995.

Casanova, H., Giersch, A., Legrand, A., Quinson, M., Suter, F., 2014. Versatile, scalable, and accurate simulation of distributed applications and platforms. Journal of Parallel and Distributed Computing 74, 2899–2917. doi:10.1016/j.jpdc.2014.06.008.

Chun, B.G., Ihm, S., Maniatis, P., Naik, M., Patti, A., 2011. CloneCloud:

729 Elastic execution between mobile device and cloud, in: Proceedings of the
730 Sixth Conference on Computer Systems (EuroSys), pp. 301–314. doi:10.1
731 145/1966445.1966473.

732 Cuervo, E., Balasubramanian, A., Cho, D.k., Wolman, A., Saroiu, S., Chandra,
733 R., Bahl, P., 2010. MAUI: Making smartphones last longer with code offload,
734 in: Proceedings of the 8th International Conference on Mobile Systems,
735 Applications, and Services (MobiSys), pp. 49–62. doi:10.1145/1814433.
736 1814441.

737 Douceur, J.R., 2002. The Sybil attack, in: Peer-to-Peer Systems: First
738 International Workshop (IPTPS 2002), Springer. pp. 251–260. doi:10.100
739 7/3-540-45748-8_24.

740 Douillard, A., Feng, Q., Rusu, A.A., Chhaparia, R., Donchev, Y., Kuncoro,
741 A., Ranzato, M., Szlam, A., Shen, J., 2023. DiLoCo: Distributed low-
742 communication training of language models. ArXiv:2311.08105.

743 Duan, J., Song, Z., Miao, X., Xi, X., Lin, D., Xu, H., Zhang, M., Jia, Z.,
744 2024. Parcae: Proactive, liveput-optimized DNN training on preemptible
745 instances, in: 21st USENIX Symposium on Networked Systems Design and
746 Implementation (NSDI), USENIX Association. pp. 1121–1139.

747 Durrani, M.N., Shamsi, J.A., 2014. Volunteer computing: Requirements,
748 challenges, and solutions. Journal of Network and Computer Applications
749 39, 369–380. doi:10.1016/j.jnca.2013.07.006.

750 exo labs, 2024. exo: Run your own AI cluster at home with everyday devices.
751 https://github.com/exo-explore/exo.

Ferreira, D., Dey, A.K., Kostakos, V., 2011. Understanding human-smartphone concerns: A study of battery life, in: Pervasive Computing (Pervasive 2011), Springer. pp. 19–33. doi:10.1007/978-3-642-21726-5_2.

Frantar, E., Ashkboos, S., Hoefler, T., Alistarh, D., 2023. GPTQ: Accurate post-training quantization for generative pre-trained transformers, in: International Conference on Learning Representations (ICLR). ArXiv:2210.17323.

Gerganov, G., contributors, 2023. llama.cpp: LLM inference in C/C++. https://github.com/ggml-org/llama.cpp.

Ghodsi, Z., Gu, T., Garg, S., 2017. SafetyNets: Verifiable execution of deep neural networks on an untrusted cloud, in: Advances in Neural Information Processing Systems (NeurIPS), pp. 4672–4681.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., et al., 2024. The Llama 3 herd of models. URL: https://arxiv.org/abs/2407.21783. arXiv:2407.21783.

Gupta, H., Vahid Dastjerdi, A., Ghosh, S.K., Buyya, R., 2017. iFogSim: A toolkit for modeling and simulation of resource management techniques in the Internet of Things, edge and fog computing environments. Software: Practice and Experience 47, 1275–1296. doi:10.1002/spe.2509.

Gupta, O., Raskar, R., 2018. Distributed learning of deep neural network over multiple agents. Journal of Network and Computer Applications 116, 1–8. doi:10.1016/j.jnca.2018.05.003.

He, Z., Zhang, T., Lee, R.B., 2019. Model inversion attacks against collaborative inference, in: Proceedings of the 35th Annual Computer Security Applications Conference (ACSAC), pp. 148–162. doi:10.1145/3359789.3359824.

Huang, Y., Cheng, Y., Bapna, A., Firat, O., Chen, M.X., Chen, D., Lee, H., Ngiam, J., Le, Q.V., Wu, Y., Chen, Z., 2019. GPipe: Efficient training of giant neural networks using pipeline parallelism, in: Advances in Neural Information Processing Systems (NeurIPS), Curran Associates.

Huba, D., Nguyen, J., Malik, K., Zhu, R., Rabbat, M., Yousefpour, A., Wu, C.J., Zhan, H., Ustinov, P., Srinivas, H., Wang, K., Shoumikhin, A., Min, J., Malek, M., 2022. Papaya: Practical, private, and scalable federated learning, in: Proceedings of Machine Learning and Systems (MLSys).

Jang, I., Yang, Z., Zhang, Z., Jin, X., Chowdhury, M., 2023. Oobleck: Resilient distributed training of large models using pipeline templates, in: Proceedings of the 29th Symposium on Operating Systems Principles (SOSP), ACM. doi:10.1145/3600006.3613152.

Javadi, B., Kondo, D., Vincent, J.M., Anderson, D.P., 2011. Discovering statistical models of availability in large distributed systems: An empirical study of SETI@home. IEEE Transactions on Parallel and Distributed Systems 22, 1896–1903. doi:10.1109/TPDS.2011.50.

Jiang, Y., Yan, R., Yao, X., Zhou, Y., Chen, B., Yuan, B., 2023. HexGen: Generative inference of large language model over heterogeneous environment. arXiv:2311.11514. arXiv:2311.11514.

Kairouz, P., McMahan, H.B., et al., 2021. Advances and open problems in federated learning. Foundations and Trends in Machine Learning 14, 1–210. doi:10.1561/2200000083.

Kondo, D., Javadi, B., Iosup, A., Epema, D., 2010. The Failure Trace Archive: Enabling comparative analysis of failures in diverse distributed systems, in: 2010 10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing (CCGrid), pp. 398–407. doi:10.1109/CCGRID.2010.71.

Kondo, D., Taufer, M., Brooks, C.L., Casanova, H., Chien, A.A., 2004. Characterizing and evaluating desktop grids: An empirical study, in: Proceedings of the 18th International Parallel and Distributed Processing Symposium (IPDPS), pp. 26–35. doi:10.1109/IPDPS.2004.1302936.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I., 2023. Efficient memory management for large language model serving with PagedAttention, in: Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP), ACM.

Lai, F., Dai, Y., Singapuram, S.S., Liu, J., Zhu, X., Madhyastha, H.V., Chowdhury, M., 2022. FedScale: Benchmarking model and system performance of federated learning at scale, in: Proceedings of the 39th International Conference on Machine Learning (ICML), PMLR.

Lai, F., Zhu, X., Madhyastha, H.V., Chowdhury, M., 2021. Oort: Efficient federated learning via guided participant selection, in: 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21), USENIX Association. pp. 19–35.

Lamport, L., Shostak, R., Pease, M., 1982. The Byzantine generals problem. ACM Transactions on Programming Languages and Systems 4, 382–401. doi:10.1145/357172.357176.

Laskaridis, S., Katevas, K., Minto, L., Haddadi, H., 2024. MELTing point: Mobile evaluation of language transformers, in: Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom), pp. 890–907. doi:10.1145/3636534.3690668.

Leviathan, Y., Kalman, M., Matias, Y., 2023. Fast inference from transformers via speculative decoding, in: Proceedings of the 40th International Conference on Machine Learning (ICML), PMLR.

Li, X., Lu, Z., Cai, D., Ma, X., Xu, M., 2024. Large language models on mobile devices: Measurements, analysis, and insights, in: Proceedings of the Workshop on Edge and Mobile Foundation Models (EdgeFM), pp. 1–6. doi:10.1145/3662006.3662059.

Li, Z., Li, T., Feng, W., Xiao, R., She, J., Huang, H., Guizani, M., Yu, H., Ho, Q., Xiang, W., Liu, X., 2026. Prima.cpp: Fast 30–70B LLM inference on heterogeneous and low-resource home clusters, in: International Conference on Learning Representations (ICLR), OpenReview.net.

Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.M., Wang, W.C., Xiao, G., Dang, X., Gan, C., Han, S., 2024. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration, in: Proceedings of Machine Learning and Systems (MLSys), pp. 87–100.

Liu, Z., Zhao, C., Iandola, F., Lai, C., Tian, Y., Fedorov, I., Xiong, Y., Chang, E., Shi, Y., Krishnamoorthi, R., Lai, L., Chandra, V., 2024. MobileLLM: Optimizing sub-billion parameter language models for on-device use cases, in: Proceedings of the 41st International Conference on Machine Learning (ICML), PMLR.

Luccioni, S., Jernite, Y., Strubell, E., 2024. Power hungry processing: Watts driving the cost of AI deployment?, in: Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT), pp. 85–99. doi:10.1145/3630106.3658542.

Mao, J., Chen, X., Nixon, K.W., Krieger, C., Chen, Y., 2017. MoDNN: Local distributed mobile computing system for deep neural network, in: Design, Automation & Test in Europe Conference (DATE), IEEE.

Mao, Z., Xia, T., Wu, Z., Chiang, W.L., Griggs, T., Bhardwaj, R., Yang, Z., Shenker, S., Stoica, I., 2025. SkyServe: Serving AI models across regions and clouds with spot instances, in: Proceedings of the Twentieth European Conference on Computer Systems (EuroSys), pp. 159–175. doi:10.1145/3689031.3717459.

McMahan, H.B., Moore, E., Ramage, D., Hampson, S., Agüera y Arcas, B., 2017. Communication-efficient learning of deep networks from decentralized data, in: Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), PMLR.

Mei, Y., Zhuang, Y., Miao, X., Yang, J., Jia, Z., Vinayak, R., 2025. Helix: Serving large language models over heterogeneous GPUs and network via

max-flow, in: Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 1, pp. 586–602. doi:10.1145/3669940.3707215.

Miao, X., Shi, C., Duan, J., Xi, X., Lin, D., Cui, B., Jia, Z., 2024. SpotServe: Serving generative large language models on preemptible instances, in: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), ACM.

MLC team, 2023. MLC-LLM. https://github.com/mlc-ai/mlc-llm. Software.

Narayanan, D., Harlap, A., Phanishayee, A., Seshadri, V., Devanur, N.R., Ganger, G.R., Gibbons, P.B., Zaharia, M., 2019. PipeDream: Generalized pipeline parallelism for DNN training, in: Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP), pp. 1–15. doi:10.1145/3341301.3359646.

Ong, J.M., Di Ferrante, M., Pazdera, A., Garner, R., Jaghouar, S., Basra, M., Ryabinin, M., Hagemann, J., 2025. TOPLOC: A locality sensitive hashing scheme for trustless verifiable inference, in: Proceedings of the 42nd International Conference on Machine Learning (ICML), PMLR. pp. 47196–47211.

Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, Í., Maleki, S., Bianchini, R., 2024. Splitwise: Efficient generative LLM inference using phase splitting,

in: 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA), pp. 118–132. doi:10.1109/ISCA59077.2024.00019.

Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Heek, J., Xiao, K., Agrawal, S., Dean, J., 2023. Efficiently scaling transformer inference, in: Proceedings of Machine Learning and Systems (MLSys), pp. 606–624.

Qu, G., Chen, Q., Wei, W., Lin, Z., Chen, X., Huang, K., 2025. Mobile edge intelligence for large language models: A contemporary survey. IEEE Communications Surveys & Tutorials 27, 3820–3860. doi:10.1109/COMST.2025.3527641.

Ryabinin, M., Dettmers, T., Diskin, M., Borzunov, A., 2023. SWARM parallelism: Training large models can be surprisingly communication-efficient, in: Proceedings of the 40th International Conference on Machine Learning (ICML), PMLR.

Ryabinin, M., Gusev, A., 2020. Towards crowdsourced training of large neural networks using decentralized mixture-of-experts, in: Advances in Neural Information Processing Systems (NeurIPS), Curran Associates.

Samsi, S., Zhao, D., McDonald, J., Li, B., Michaleas, A., Jones, M., Bergeron, W., Kepner, J., Tiwari, D., Gadepally, V., 2023. From words to watts: Benchmarking the energy costs of large language model inference, in: 2023 IEEE High Performance Extreme Computing Conference (HPEC), pp. 1–9. doi:10.1109/HPEC58863.2023.10363447.

Sheng, Y., Zheng, L., Yuan, B., Li, Z., Ryabinin, M., Chen, B., Liang, P., Ré, C., Stoica, I., Zhang, C., 2023. FlexGen: High-throughput generative

inference of large language models with a single GPU, in: Proceedings of the 40th International Conference on Machine Learning (ICML), PMLR.

Shirts, M., Pande, V.S., 2000. Screen savers of the world unite! Science 290, 1903–1904. doi:10.1126/science.290.5498.1903.

Shoeybi, M., Patwary, M., Puri, R., LeGresley, P., Casper, J., Catanzaro, B., 2019. Megatron-LM: Training multi-billion parameter language models using model parallelism. URL: https://arxiv.org/abs/1909.08053. arXiv:1909.08053.

Sun, H., Li, J., Zhang, H., 2024. zkLLM: Zero knowledge proofs for large language models, in: Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 4405–4419. doi:10.1145/3658644.3670334.

Thorpe, J., Zhao, P., Eyolfson, J., Qiao, Y., Jia, Z., Zhang, M., Netravali, R., Xu, G.H., 2023. Bamboo: Making preemptible instances resilient for affordable training of large DNNs, in: 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI), USENIX Association.

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., et al., 2023. Llama 2: Open foundation and fine-tuned chat models. URL: https://arxiv.org/abs/2307.09288. arXiv:2307.09288.

Tramèr, F., Boneh, D., 2019. Slalom: Fast, verifiable and private execution of neural networks in trusted hardware, in: International Conference on Learning Representations (ICLR). ArXiv:1806.03287.

Vetter, J., Novák, P., Wagner, M.R., Veit, C., Möller, K.C., Besenhard, J.O., Winter, M., Wohlfahrt-Mehrens, M., Vogler, C., Hammouche, A., 2005. Ageing mechanisms in lithium-ion batteries. Journal of Power Sources 147, 269–281. doi:10.1016/j.jpowsour.2005.01.006.

Wu, L., Liu, X., Shi, T., Ye, Z., Song, D., 2025. DeServe: Towards affordable offline LLM inference via decentralization. arXiv:2501.14784. arXiv:2501.14784.

Xue, Z., Song, Y., Mi, Z., Zheng, X., Xia, Y., Chen, H., 2024. PowerInfer-2: Fast large language model inference on a smartphone. ArXiv:2406.06282.

Yang, C., Wang, Q., Xu, M., Chen, Z., Bian, K., Liu, Y., Liu, X., 2021. Characterizing impacts of heterogeneity in federated learning upon large-scale smartphone data, in: Proceedings of the Web Conference (WWW), ACM.

Ye, S., Du, J., Zeng, L., Ou, W., Chu, X., Lu, Y., Chen, X., 2024. Galaxy: A resource-efficient collaborative edge AI system for in-situ transformer inference, in: Proceedings of IEEE INFOCOM, IEEE. ArXiv:2405.17245.

Yu, G.I., Jeong, J.S., Kim, G.W., Kim, S., Chun, B.G., 2022. Orca: A distributed serving system for transformer-based generative models, in: 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI), USENIX Association.

Zhan, Y., Zhang, J., Hong, Z., Wu, L., Li, P., Guo, S., 2021. A survey of incentive mechanism design for federated learning. IEEE Transactions on Emerging Topics in Computing doi:10.1109/TETC.2021.3063517.

Zhang, M., Cao, J., Shen, X., Cui, Z., 2024. EdgeShard: Efficient LLM inference via collaborative edge computing. ArXiv:2405.14371.

Zhao, J., Song, Y., Liu, S., Harris, I.G., Jyothi, S.A., 2023. LinguaLinked: A distributed large language model inference system for mobile devices. ArXiv:2312.00388.

Zhao, Z., Barijough, K.M., Gerstlauer, A., 2018. DeepThings: Distributed adaptive deep learning inference on resource-constrained IoT edge clusters. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 37.

Zheng, L., Li, Z., Zhang, H., Zhuang, Y., Chen, Z., Huang, Y., Wang, Y., Xu, Y., Zhuo, D., Xing, E.P., Gonzalez, J.E., Stoica, I., 2022. Alpa: Automating inter- and intra-operator parallelism for distributed deep learning, in: 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22), USENIX Association. pp. 559–578.

Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., Zhang, H., 2024. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving, in: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), USENIX Association. pp. 193–210.

Table 2: Shard traffic predicted by the analytical model against the simulator (GB of model blocks downloaded per million delivered tokens; activations excluded). Downloads are split into those at pipeline formation and those at repair or refill (counts per run, simulator / model). Means over seeds.

Policy	Phones	Sim.	Model	Error	Formation	Repair
Restart	1,000	375.2	386.9	+3.1%	439 / 415	889 / 955
Replay	1,000	361.9	373.3	+3.1%	439 / 415	889 / 955
Cache-aware	1,000	316.2	335.6	+6.1%	282 / 289	896 / 961
Standby	1,000	442.8	446.3	+0.8%	244 / 211	738 / 778
Lifetime	1,000	560.5	564.7	+0.7%	737 / 755	1227 / 1224
Consolidate	1,000	323.4	349.4	+8.0%	501 / 511	781 / 873
Wait	1,000	251.2	274.3	+9.2%	152 / 165	654 / 716
Sticky	1,000	247.4	277.1	+12.0%	154 / 174	654 / 731
NightShift, no wait	1,000	273.5	291.6	+6.6%	187 / 196	754 / 808
NightShift	1,000	30.3	29.6	-2.5%	28 / 27	85 / 84
Replay	500	331.5	339.9	+2.5%	217 / 209	403 / 426
Cache-aware	500	287.5	307.2	+6.8%	157 / 155	384 / 425
Consolidate	500	306.3	326.3	+6.5%	241 / 247	365 / 398
NightShift	500	130.1	126.8	-2.5%	60 / 46	157 / 167
Replay	2,000	365.5	377.6	+3.3%	925 / 872	1804 / 1948
Cache-aware	2,000	311.3	335.2	+7.7%	545 / 587	1817 / 1956
Consolidate	2,000	312.8	333.1	+6.5%	971 / 948	1569 / 1756
NightShift	2,000	11.9	13.4	+12.2%	20 / 27	79 / 84
Replay	4,000	378.9	390.7	+3.1%	1892 / 1823	3805 / 4052
Cache-aware	4,000	314.4	340.2	+8.2%	1031 / 1136	3766 / 4055
Consolidate	4,000	311.4	331.9	+6.6%	1838 / 1800	3263 / 3636
NightShift	4,000	5.1	4.8	-5.3%	18 / 20	72 / 65

Table 3: Simulator parameters. Values are assumptions; Section 9 reports the measured values that replace them.

Model	80 layers × 0.5 GB, KV 4 KiB/token/layer
Workload	batches of 32 seq., 512 prompt + 256 output tokens
Prefill cost	1/8 of a decode token
Phones	RAM, share of pool, ms per layer: 6 GB, 10%, 30; 8 GB, 40%, 18; 12 GB, 35%, 12; 16 GB, 15%, 10 (log-normal spread σ=0.2)
Usable RAM	45% of physical
Network	60 ms per hop; downlink median 50 Mbit/s
Flash read	1 GB/s
Power	4 W computing, 1 W waiting
Night charging	85% of nights, start 23:00±1 h, 7±1.5 h
Day charging	1.5 sessions/day, median 35 min, 60% on Wi-Fi
Flash cache	1 block (2.5 GB) per phone; Section 8 varies it to 16
Activations	16 KiB per token per sequence, sent on repair
Timers	home-block wait 300 s; vacant pipeline dissolved after 600 s
Warm-up	2 days run and discarded before measuring